

ENGINEERING REFERENCE

Primer: Loop Engineering for Agentic Systems

Designing bounded observe–plan–act–verify cycles that stop safely

Primer takeaway A loop is a controlled state machine, not permission to keep trying. Every iteration must have admitted evidence, a legal action, verification, a budget, a continuation guard, and an explicit stop condition.

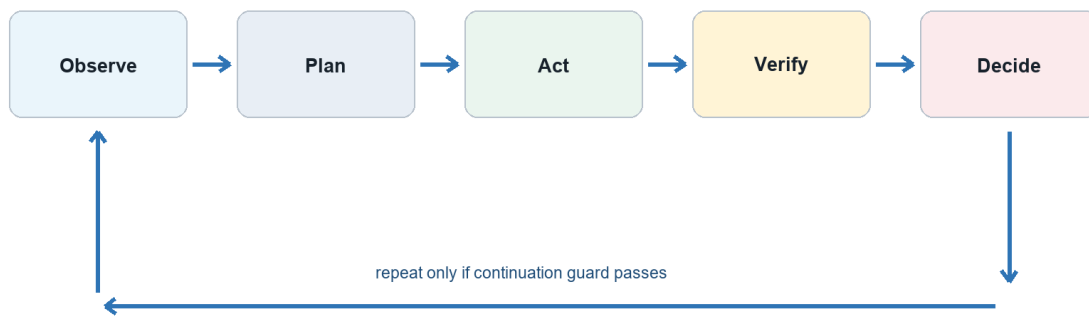
How to use this handout

Read it once end to end, then keep the checklists and templates beside the agent specification while authoring or reviewing.

What loop engineering is

Loop engineering designs repeated agent behavior as an observable control system. It defines the state carried between iterations, the evidence admitted at each step, the tools and actions permitted, the verification required, the conditions for another iteration, and the conditions that end or escalate the run.

A safe loop repeats only while evidence, guards, and budgets permit



Stop conditions: objective satisfied • evidence insufficient • protected invariant fails • budget exhausted • human gate required

The five-step cycle

Step	Engineering question
Observe	What changed? Which evidence is new, trustworthy, and relevant?
Plan	What bounded action is allowed by policy, state, scope, and budget?
Act	Which tool call or transformation is executed, with what immutable inputs?
Verify	Did the action satisfy the expected invariant without protected regression?

Step	Engineering question
Decide	Stop, repeat, retry, fail, emit incomplete_input, or request human action?

Repository basis: orchestration/workflow-model.md; governance/reviewer-calibration-and-evolution.md

The loop control model

Control	Must define	Evidence retained
Objective	A measurable completion condition	Objective ID and acceptance rule
State	Durable lifecycle state and iteration state	State before and after
Observation	Admissible evidence and freshness rules	Evidence IDs, hashes, coverage
Policy	Allowed actions, tools, permissions, and boundaries	Policy and contract versions
Action	One bounded mutation or analysis step	Tool input, output, execution ID
Verification	Expected invariant and regression checks	Test results and score deltas
Budget	Iteration, time, token, tool, cost, and mutation limits	Consumption and remaining budget
Termination	Success, failure, insufficient evidence, budget, or human gate	Stop reason and final state

Continuation guard Repeat only when the objective is unmet, the next action is legal and materially different, verification supports continuation, protected invariants hold, and sufficient budget remains.

Design the loop before writing the prompt

1. Define the objective and terminal success evidence.
2. Name the lifecycle states and legal transitions.
3. Define the observation contract: sources, freshness, provenance, coverage, and conflict handling.
4. Define the action space: permitted tools, mutations, credentials, side effects, and reversibility.
5. Define verification: expected invariants, tests, validators, and consumer compatibility.
6. Set budgets: maximum iterations, elapsed time, tokens, calls, cost, mutations, and tolerated regressions.
7. Define retry policy separately from iteration policy, including backoff and retryable error classes.
8. Define stop and escalation conditions, including the exact human gate.
9. Specify audit events and the state snapshot retained after every iteration.

Iteration is not retry

Mechanism	Purpose	Example
Iteration	Uses verified learning to choose a materially informed next action.	Revise a candidate prompt after benchmark evidence.
Retry	Repeats the same logical action after a transient failure.	Repeat a timed-out read-only tool call with backoff.
Replan	Changes the action sequence because state or evidence changed.	Select another admitted evidence path after a source becomes unavailable.

Controls that keep loops safe

Control	Implementation guidance
Idempotency	Use stable operation IDs; repeated execution must not duplicate irreversible effects.
Checkpointing	Persist state, evidence lineage, budget, pending actions, and stop reason after each iteration.
Least privilege	Grant only the tools and credentials required for the current state and action.
Fail closed	Stop on identity, integrity, contract, policy, or source-revision ambiguity.
Bounded retry	Retry only declared transient failures; use attempt caps, backoff, and timeout.
Progress test	Require measurable new evidence, reduced uncertainty, or improved benchmark result.
Regression guard	Block continuation when a protected invariant or material compatibility condition fails.
Human gate	Pause before approval, risk acceptance, promotion, irreversible mutation, or policy exception.
Deterministic audit	Record state transitions, inputs, actions, outputs, versions, checks, and budget use.

Repository basis: [contracts/orchestration-agent-contract.md](#); [contracts/universal-agent-contract.md](#)

Common loop failure modes

Failure mode	What it looks like	Required hardening
Infinite loop	No terminal condition or progress test	Iteration cap, elapsed-time cap, progress invariant
Thrashing	The loop alternates between states without learning	Cycle detection, state hash, repeated-plan guard
Goal drift	A later iteration broadens or changes the objective	Immutable objective ID and scope check each cycle
Evidence laundering	Prior model output becomes unquestioned evidence	Evidence admission and provenance validation
Retry storm	Transient failures trigger unbounded repeated calls	Retry classes, exponential backoff, circuit breaker
Duplicate side effects	The same action executes more than once	Idempotency key, deduplication, transactional boundary
Budget blindness	The loop spends tokens, time, or money without visibility	Budget ledger and pre-action budget guard
Premature success	The model declares completion without verification	Independent validator and explicit success evidence
Silent partial result	A stopped loop presents incomplete work as complete	Explicit incomplete_input or failed terminal state
Authority leak	The loop promotes or approves its own candidate	Separate human review and promotion decision

Loop contract template

Field	Declaration
loop_id / version	Stable identity and semantic version
objective	Owned outcome and terminal success evidence
initial_state	Required starting lifecycle state and prerequisites
observation_contract	Evidence types, provenance, freshness, coverage, conflicts
action_policy	Permitted tools, mutations, permissions, side-effect class
verification_contract	Validators, invariants, regression and compatibility checks
continuation_guard	Conditions that authorize another iteration
retry_policy	Retryable classes, attempts, backoff, timeout, circuit breaker
budgets	Iterations, time, tokens, calls, cost, mutations, regression tolerance
terminal_states	Succeeded, incomplete_input, failed, superseded, human_review
audit_events	State, evidence, action, verification, budget, and stop records
human_gates	Named decisions and irreversible actions requiring authority

Reference pseudocode

```
state = validate_initial_state(work_packet)
for iteration in range(max_iterations):
    evidence = observe(state)
    if not evidence_integrity_ok(evidence): stop("failed")

    plan = choose_bounded_action(state, evidence, remaining_budget)
    if not continuation_guard(state, evidence, plan): stop(reason)

    result = execute_idempotently(plan)
    verification = verify(result, protected_invariants)
    record_audit_event(state, evidence, plan, result, verification)

    if verification.success: stop("succeeded")
    if verification.requires_human: stop("human_review")
    state = transition(state, verification)

stop("budget_exhausted")
```

Worked example: controlled prompt evolution

The repository's reviewer-evolution loop is a useful model because it separates iterative candidate improvement from production authority.

Stage	Evidence-controlled behavior
Observe	A benchmark exposes weak incomplete-input handling.
Analyze	Trace the failure to a missing state clause, not to the gold label.
Propose	Create a versioned candidate prompt with the smallest corrective change.
Benchmark	Run the approved subset, then the required full suite.
Regress	Compare protected invariants, compatibility, and material score losses.
Human review	Inspect the diff, evidence, gains, regressions, and remaining uncertainty.
Promote or reject	An authorized human records disposition; production never self-modifies.

Loop review checklist

- Is the objective immutable and independently verifiable?
- Can every transition be explained by an event and a guard?
- Does each iteration produce measurable progress or stop?
- Are iteration, retry, and replan behaviors distinct?
- Are side effects idempotent, reversible, or human-gated?
- Are budgets checked before action and recorded after action?
- Can partial, failed, superseded, and human-review states be distinguished?
- Does the audit record reproduce every action and decision request?

Final question If the loop stops unexpectedly, can another engineer explain exactly what happened, why it stopped, what changed, and what authority is required next?

Repository basis: governance/reviewer-calibration-and-evolution.md; orchestration/workflow-model.md